

星露谷战斗模块

俯视角像素风动作角色扮演游戏

概要设计说明书

课程名称： 软件项目开发与实践

项目名称： 星露谷战斗模块

姓 名： 谢宇鹏

学 号： 5120243390

开发模式： B 模式（AI 智能协作开发）

技术栈： Python 3.12 + Pygame 2.6

文档版本： V2.1

本文档阐述系统架构设计、模块划分、面向对象类结构和核心状态机设计。

目录

1 引言 2

1.1 编写目的 2

1.2 设计原则 2

2 系统架构设计 3

2.1 四层分层架构 3

3 模块划分 4

4 面向对象设计 6

4.1 核心类结构 6

4.2 Entity 基类设计 6

4.3 管理器组合设计 7

5 核心状态机设计 8

5.1 游戏主流程状态机 8

5.2 敌人 AI 有限状态机 8

5.3 Boss 三阶段状态机 10

6 武器与魔法系统 12

6.1 武器系统 12

6.2 魔法系统 12

7 数据结构设计 14

7.1 地图数据 14

7.2 配置数据 14

7.3 存档数据 14

8 关键设计决策 15

8.1 A* 寻路算法选型 15

8.2 分轴碰撞检测 15

8.3 Y 轴排序渲染 15

8.4 数据驱动设计 15

9 项目概况 16

参考文献

16

引言

编写目的

本文档是「星露谷战斗模块」的概要设计说明书，描述系统的整体架构、模块划分、核心类的设计以及关键状态机的设计。本文档是连接需求分析与编码实现的桥梁。

设计原则

原则	说明
分层架构	系统分为控制层、逻辑层、实体层、数据层，上层依赖下层
单一职责	每个类只负责一个功能域，如战斗、Boss、地图各由独立 Manager 管理
合成复用	Level 通过组合模式整合 CombatManager、BossManager 等组件
数据驱动	武器、怪物、魔法的属性全部集中在 settings.py 中配置，新增类型只需加数据
开闭原则	对扩展开放（加配置即可新增怪物/武器），对修改关闭（不需改核心逻辑）

系统架构设计

四层分层架构

系统采用分层架构，分为四个层次：

层级	职责	核心模块
控制层	游戏主循环、状态机、输入事件、界面渲染	Game, HUD, Upgrade, Death-Screen, VictoryScreen
逻辑层	关卡管理、战斗调度、Boss 生命周期、地图加载	Level, CombatManager, BossManager, MapManager
实体层	玩家、敌人、武器、魔法、粒子、瓦片	Player, Enemy, Weapon, Magic-Player, Tile
数据层	全局配置、资源缓存、寻路算法、存档	Settings, ResourceManager, Pathfinding, SaveManager

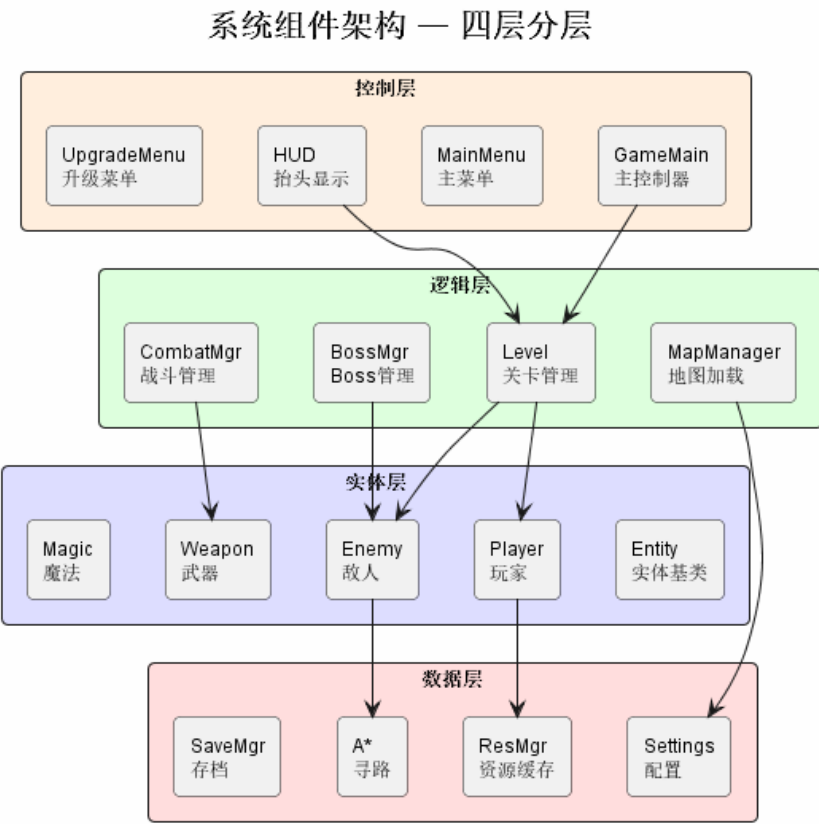


图 1: 系统组件架构图

模块划分

系统由多个 Python 源文件组成，按职责划分为若干逻辑包：

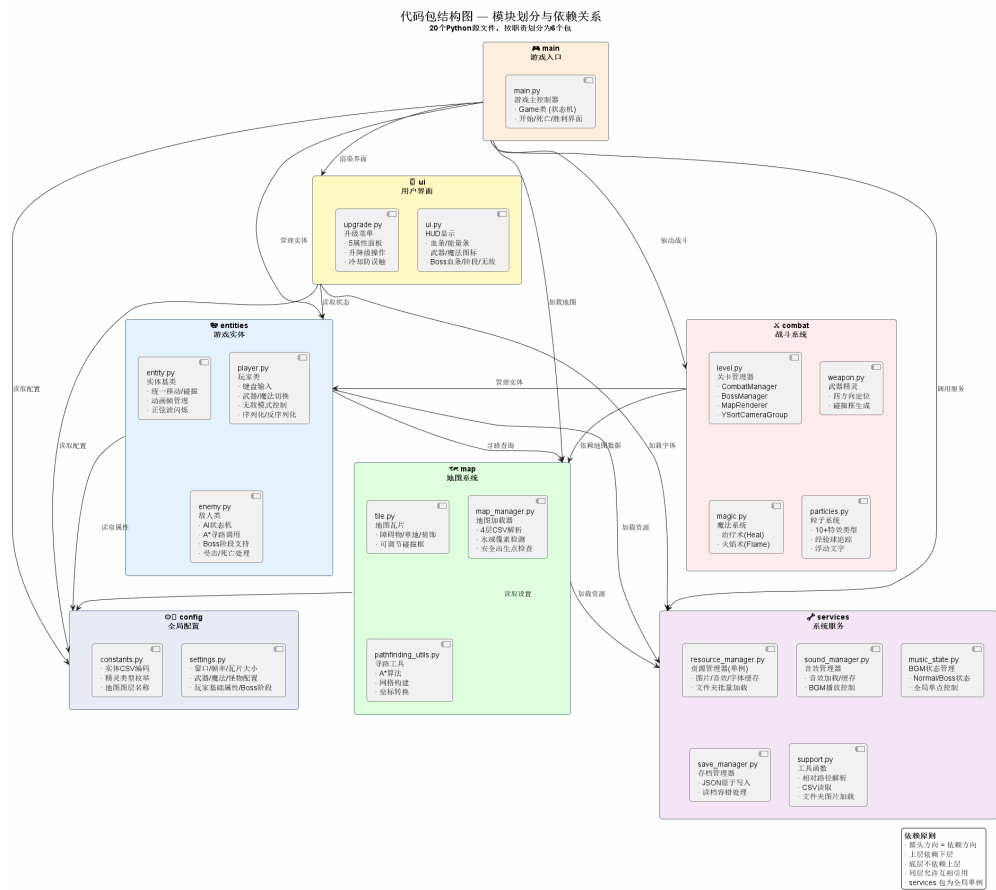


图 2: 代码包结构图

各包职责如下：

- **main（入口）**：游戏主控制器，管理全局状态机（主菜单/游戏/死亡/胜利）
- **config（配置）**：集中管理窗口参数、武器属性、怪物属性、Boss 阶段等配置数据
- **entities（实体）**：Entity 实体基类（移动/碰撞/动画）、Player（键盘驱动）、Enemy（AI 驱动）
- **map（地图）**：CSV 瓦片地图解析、水域检测、A* 寻路算法与导航网格构建
- **combat（战斗）**：关卡协调器 Level、CombatManager 战斗管理、BossManager 阶段管理、武器精灵、魔法系统、粒子特效

- **ui (界面):** HUD 显示 (血条/能量/武器/Boss 血条)、升级菜单 (五维属性分配)
- **services (服务):** 资源缓存单例、音效管理、BGM 状态控制、JSON 存档

面向对象设计

核心类结构

系统的类结构分为以下层次:

- **实体继承层次:** Entity (基类) 定义移动、碰撞、动画的公共行为; Player (玩家) 由键盘/鼠标驱动; Enemy (敌人) 由 AI 状态机驱动
- **管理器组合层次:** Level (关卡协调器) 通过组合模式集成 CombatManager (战斗调度)、BossManager (Boss 生命周期)、MapRenderer (世界地图)
- **武器与魔法:** Weapon 为短暂存在的精灵, MagicPlayer 根据配置数据驱动火焰和治疗两种魔法
- **UI 与数据:** UI 负责 HUD 渲染, Upgrade 负责升级交互, ResourceManager (单例) 负责资源缓存

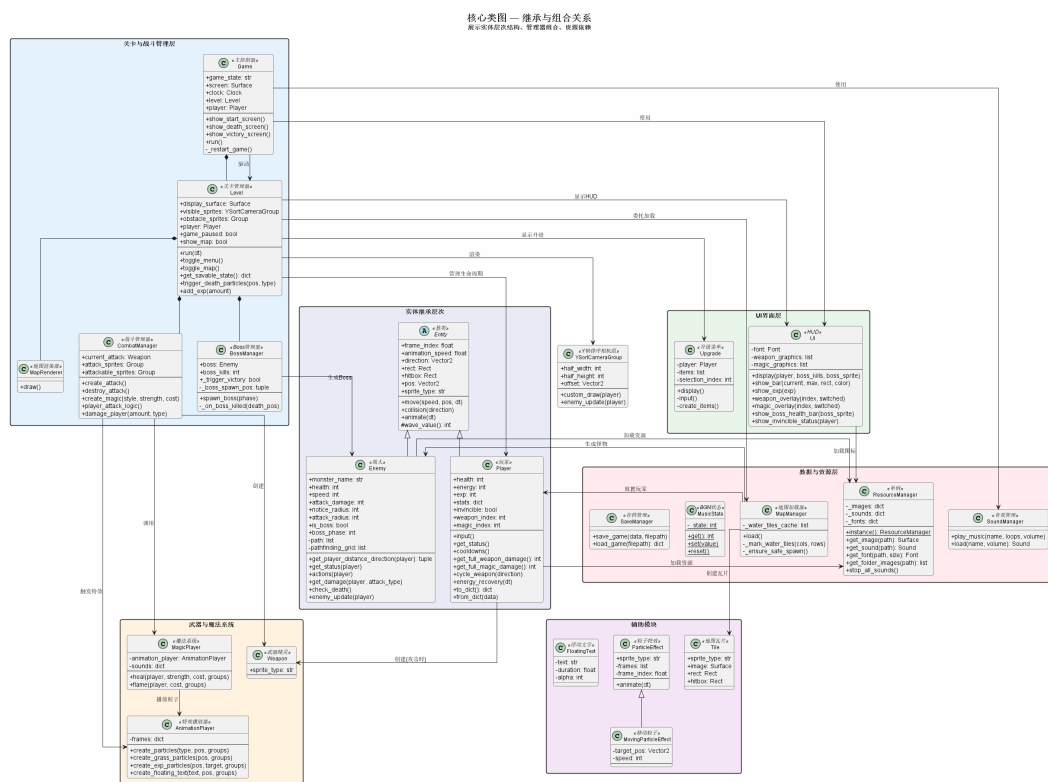


图 3: 核心类图——继承与组合关系

Entity 基类设计

Entity 是所有游戏实体的抽象基类，提供三个核心能力：

1. **统一移动**: $\text{移动量} = \text{速度} \times \text{方向向量} \times \text{帧时间} (\text{delta time})$, 保证不同帧率下移动速度一致
2. **分轴碰撞检测**: 先水平移动并检测碰撞修正, 再垂直移动并检测碰撞修正, 从根本上防止对角线穿墙
3. **动画帧管理**: 帧索引以浮点数累加 ($\text{帧索引} += \text{动画速度} \times \text{帧时间}$), 支持不同帧数的动画无缝混合

Player 和 Enemy 继承 Entity 后, 各自重写 input/actions 方法实现不同的控制方式 (键盘驱动 vs AI 驱动)。

管理器组合设计

Level 作为核心协调者, 通过组合模式整合三个专职管理器:

- **CombatManager**: 管理武器/魔法的创建销毁、攻击精灵碰撞判定、玩家受伤中转
- **BossManager**: 管理 Boss 完整生命周期——在 CSV 标记位置生成、被击杀后晋升下一阶段 (属性倍增 + 颜色变化)、三阶段全部击杀触发胜利
- **MapRenderer**: 全屏世界地图渲染——障碍网格、敌人红点、Boss 红叉脉冲标记、玩家绿点、实时坐标

核心状态机设计

游戏主流程状态机

游戏运行时的全局状态机包含六个状态，由玩家操作和游戏事件驱动转换：

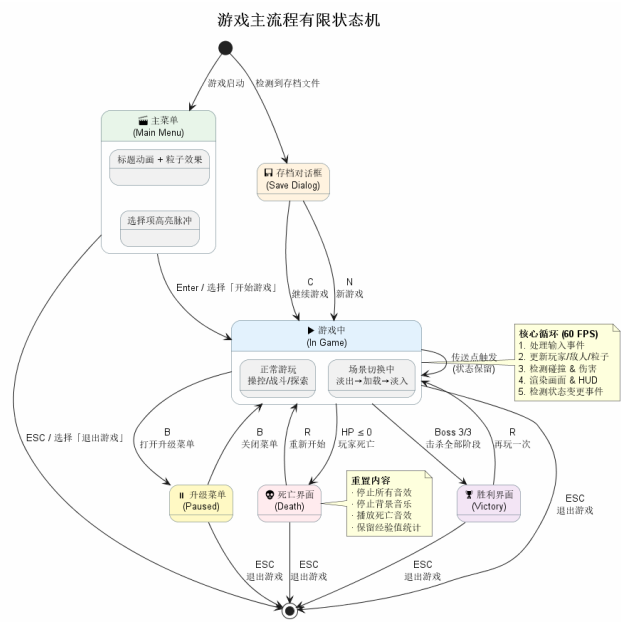


图 4: 游戏主流程状态机

状态说明：

- **主菜单：**游戏启动后显示，支持「开始游戏」和「退出游戏」选择，检测到存档时弹出继续/新游戏对话框
- **游戏中：**核心游戏循环，60FPS 更新玩家/敌人/武器/魔法/粒子
- **升级菜单：**暂停状态，覆盖半透明菜单，分配经验值到五维属性
- **死亡界面：**HP 归零时进入，显示经验统计，可选重开或退出
- **胜利界面：**Boss 三阶段全部击杀后进入

敌人 AI 有限状态机

四种怪物共用同一状态机，通过不同参数（警戒范围、攻击范围、伤害值、速度）实现差异化行为：

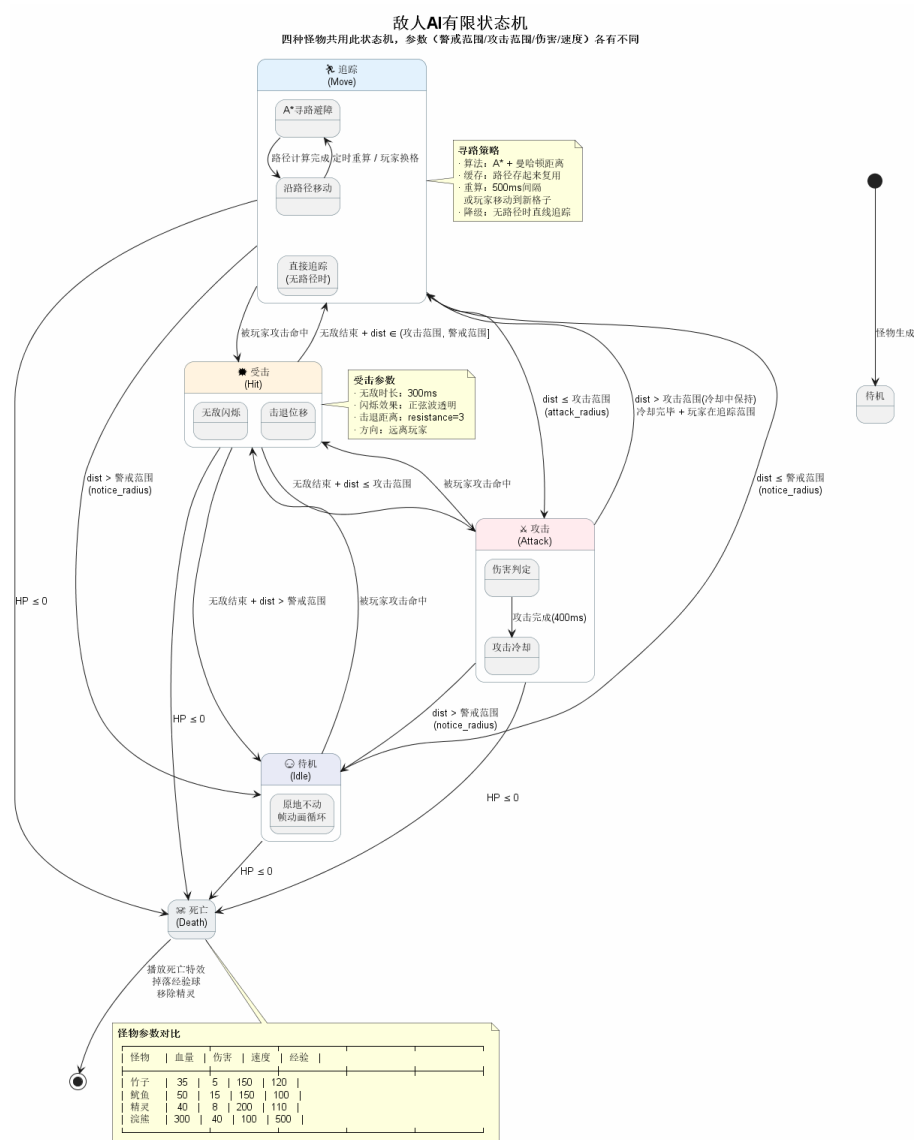


图 5: 敌人 AI 有限状态机

五种状态及其转换条件：

1. 待机（Idle）：距离 > 警戒范围时，怪物原地播放 idle 动画
2. 追踪（Move）：距离 ≤ 警戒范围时，调用 A* 算法计算绕障碍路径，沿路径移动
3. 攻击（Attack）：距离 ≤ 攻击范围时，造成伤害并进入冷却（400ms）
4. 受击（Hit）：被玩家命中后进入 300ms 无敌闪烁 + 击退效果
5. 死亡（Death）：HP 归零，播放特效，掉落经验球，移除精灵

Boss 三阶段状态机

Boss（浣熊 Raccoon）拥有三阶段形态，每击杀一次晋升下一阶段：

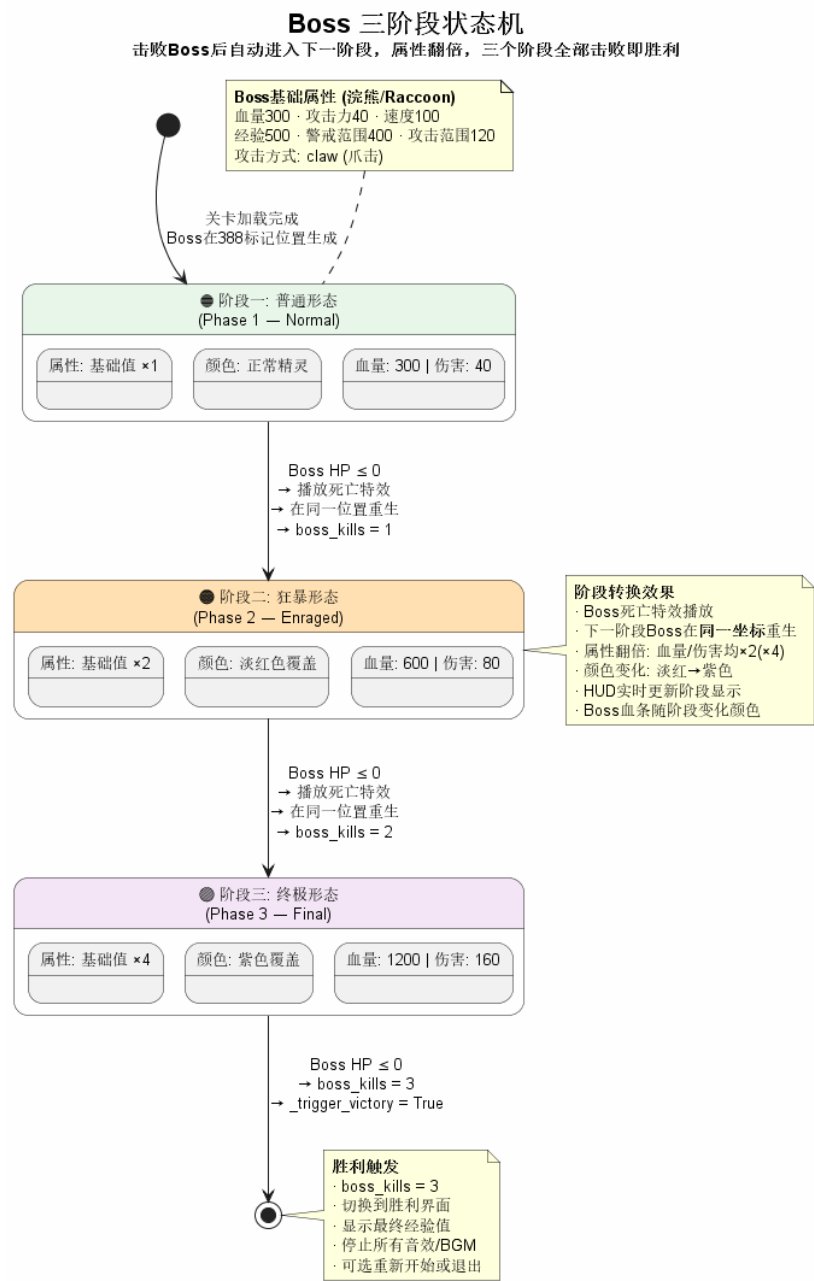


图 6: Boss 三阶段状态机

阶段设计：

- 阶段一（Normal）：基础属性，血量 300/伤害 40，正常精灵颜色
- 阶段二（Enraged）：属性 ×2，血量 600/伤害 80，淡红色覆盖

- **阶段三 (Final):** 属性 $\times 4$, 血量 1200/伤害 160, 紫色覆盖
- **胜利触发:** 三个阶段全部击杀 $\rightarrow$ boss_kills = 3 $\rightarrow$ 进入胜利界面

武器与魔法系统

系统采用配置驱动设计——所有武器和魔法的属性集中定义在 `settings.py` 中，新增武器/魔法只需添加配置条目，核心逻辑代码无需修改。

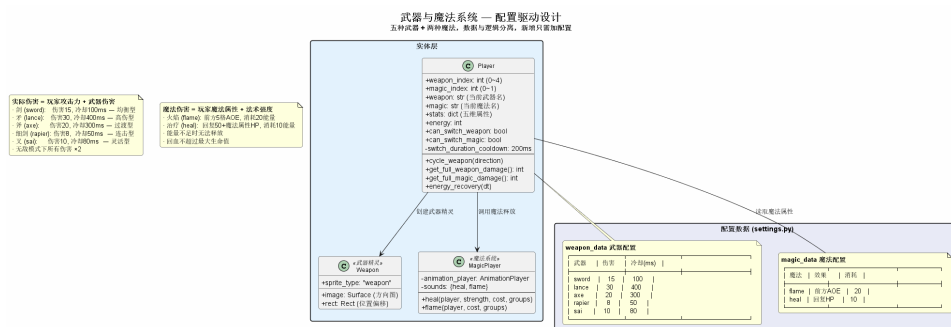


图 7: 武器与魔法系统——配置驱动设计

武器系统

玩家可通过 Q 键或鼠标滚轮在五种武器间循环切换，每次切换有 200ms 冷却防误触：

武器	伤害	冷却	特点
剑 (sword)	15	100ms	均衡，适合新手
矛 (lance)	30	400ms	单次伤害最高，攻速最慢
斧 (axe)	20	300ms	伤害和速度均中等
细剑 (rapier)	8	50ms	攻速极快，连击伤害最高
叉 (sai)	10	80ms	快速灵活

实际伤害 = 玩家攻击力属性 + 武器伤害。无敌模式下所有伤害 $\times 2$ 。

魔法系统

玩家按 E 键切换魔法类型，鼠标右键释放：

魔法	效果	消耗
治疗 (heal)	回复 HP (回复量 = 50 + 玩家魔法属性), 不超过最大血量	10 能量
火焰 (flame)	在角色前方扩散 5 个火焰粒子, 造成范围伤害	20 能量

能量自动恢复速度与魔法属性挂钩。能量不足时无法释放魔法。

数据结构设计

地图数据

每张地图由四个 CSV 文件组成，每个文件为整数二维矩阵：

文件	内容	特殊值
FloorBlocks.csv	边界与碰撞数据	-1= 空, 其他 = 障碍物
Grass.csv	草地与可破坏物	-1= 空, 其他 = 草地（随机选图）
Objects.csv	装饰物体	-1= 空, 数值 = 物体图片索引
Entities.csv	实体生成点	394= 玩 家, 392=Boss, 390/391/393= 怪物类型

配置数据

所有游戏配置数据采用字典嵌套结构集中管理：

- `weapon_data[名称]`: damage, cooldown, graphic
- `magic_data[名称]`: strength, cost, graphic, sound
- `monster_data[名称]`: health, damage, speed, exp, attack_type, notice_radius, attack_radius
- `BOSS_PHASE_DATA[阶段]`: color, stat_mult
- `PLAYER_BASE_STATS`: health=200, energy=100, attack=15, magic=5, speed=300

存档数据

JSON 格式，包含玩家状态（位置/血量/能量/经验/五维属性/武器/魔法）和世界状态（已击败敌人坐标列表、已破坏草地坐标列表）。采用原子写入机制：先写临时文件，成功后原子替换，防止存档损坏。

关键设计决策

A* 寻路算法选型

敌人追踪玩家需要绕开障碍物，对比了几种方案后选择 A* 算法：

方案	最短路径	搜索效率	结论
BFS（层层扩展）	保证	很低	太慢
Dijkstra	保证	中等	可用但效率一般
A*	保证	高	最终选择
贪心搜索	不保证	极高	路径质量差

A* 使用评估函数 $f(n) = g(n) + h(n)$ ，其中 $g(n)$ 为已走步数， $h(n)$ 为曼哈顿距离估算。为优化性能，计算好的路径缓存起来，每隔 500ms 或玩家移动到新网格时才重新计算。

分轴碰撞检测

角色移动时先处理水平方向（X 轴）的移动和碰撞修正，再处理垂直方向（Y 轴）。分轴处理从根本上避免了对角线方向同时穿过两个障碍物夹角的穿墙问题。

Y 轴排序渲染

俯视角游戏中，每帧将所有精灵按 Y 坐标升序排序后依次绘制。Y 值小（画面上方）的先画，Y 值大（画面下方）的后画，自然形成正确的遮挡关系。

数据驱动设计

武器属性、怪物属性、魔法属性、Boss 阶段数据全部集中配置在 settings.py 字典中。新增怪物类型只需添加一条配置记录和对应的动画资源，无需修改 AI 逻辑代码。这种设计使得系统具有良好的可扩展性。

项目概况

类别	规模	说明
Python 源文件	十余个	核心游戏逻辑, 多模块多文件结构
代码总规模	约两千行	包含注释和空行
地图数据	四层 CSV 文件	每 张 地 图 由 Floor-Blocks/Grass/Objects/Entities 组成
图片资源	大量 PNG 图片	角色动画帧、怪物动画帧、武器精灵、粒子特效、草地/物体/UI
音频资源	十余个音频	背景音乐 (OGG)+ 攻击/受击/死亡/魔法音效 (WAV)
存档格式	JSON	玩家状态 + 世界状态
地图格式	CSV	多层二维整数矩阵

参考文献

1. Pygame Official Documentation. <https://www.pygame.org/docs/>
2. Hart, P.E., Nilsson, N.J., Raphael, B. *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*. 1968.
3. Gamma, E., Helm, R., Johnson, R., Vlissides, J. *Design Patterns*. Addison-Wesley, 1994.
4. Stardew Valley. ConcernedApe.
5. 软件项目开发与实践课程资料. 刘畅.